

Socket in C

Per scambiare messaggi tra processi (i nostri software) potremmo usare un file, ad esempio un file di testo, questa è una soluzione che tecnicamente può funzionare ma che porta a enormi problemi di accesso concorrente e dipendenze. Allora potremmo usare dei protocolli di rete per scambiare messaggi tra processi in modo da evitare queste problematiche, questo sono le *socket*.

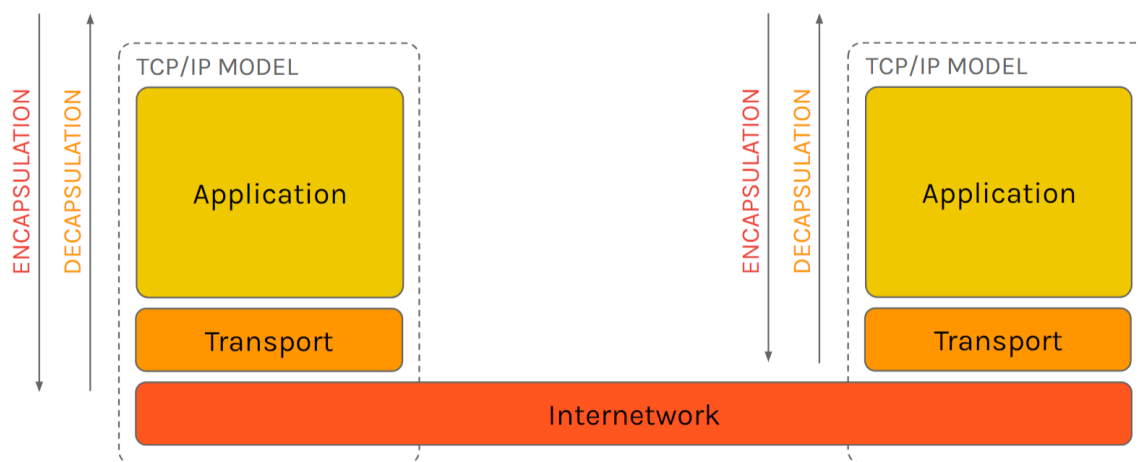
Ci sono 2 tipologie principali di socket

- Stream socket

Queste socket sono usate per comunicare come uno stream di byte in cui possiamo scrivere o leggere da uno stream nello stesso ordine in cui sono stati mandati. La frammentazione dei dati nello stream o la gestione di pacchetti persi saranno gestite direttamente dalla socket in base al protocollo utilizzato. *Le stream socket solitamente usano TCP.*

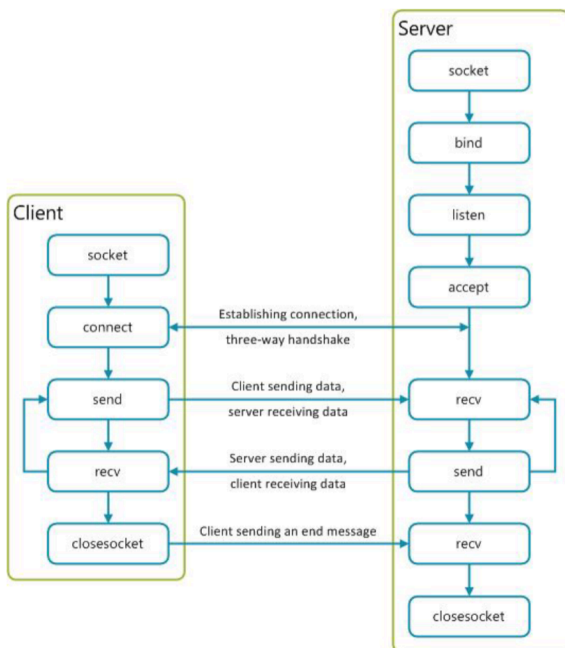
- Datagram socket

Se non abbiamo bisogno di tutti i servizi offerti dalle stream socket (gestione dei pacchetti persi, congestion control, riordinamento ecc...) perché magari abbiamo bisogno di velocità allora possiamo usare le datagram socket, queste socket inviano dati senza le funzionalità delle stream socket. *Le datagram socket solitamente utilizzano UDP.*



Le socket sono gestite tramite uno standard chiamato Berkeley (BSD socket) e questo standard sostanzialmente è una libreria scritta in C. Ogni altro linguaggio di programmazione si basa su questa libreria applicando delle traduzioni opportune per farla funzionare.

Questo è il funzionamento teorico di una socket:



Function Call	Description
Socket()	To create a socket
Bind()	It's a socket identification like a telephone number to contact
Listen()	Ready to receive a connection
Connect()	Ready to act as a sender
Accept()	Confirmation, it is like accepting to receive a call from a sender
Send()	To send data
Recv()	To receive data
Close()	To close a connection

Funzione Socket()

Solitamente si comunica tra client e server e la funzione comune tra i 2 è *socket()*

- `int socket(int domain, int type, int protocol);`

La funzione `socket()` restituisce un **numero intero**, esattamente come fa la funzione `open()` quando si apre un file di testo sul disco.

- Questo numero intero è chiamato **File Descriptor (FD)**.
- Rappresenta l'indice (un "ticket") all'interno di una tabella interna del Sistema Operativo chiamata *Tabella dei File Aperti*.
- È possibile comunicare in rete usando le funzioni standard di lettura e scrittura dei file di sistema (`read()` e `write()`), invece di quelle specifiche per le socket (`recv()` e `send()`)

Il parametro `int domain` specifica la famiglia di protocolli che vuoi usare, ne esistono molti e sono implementati nella libreria `sys/socket.h`

- Usa `PF_INET` per UDP, TCP e altri protocolli.

Per il parametro `int type` sono definiti 5 valori, tutti iniziano con "`SOCK_`" il più comune è `SOCK_STREAM` che indica che vogliamo usare una stream socket e quindi TCP, se usassimo `SOCK_DGRAM` vogliamo usare una datagram socket e quindi UDP.

Nel parametro `int protocol` noi andremo ad inserire sempre "0".

SOCKET STREAM

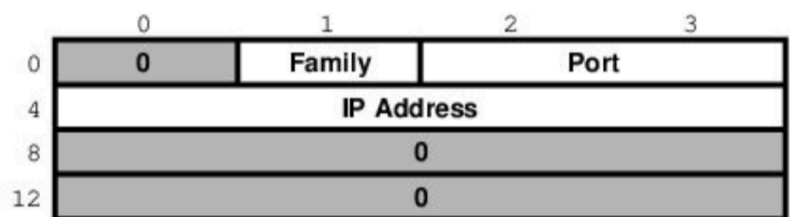
Il Problema del linguaggio C

Funzioni come `bind()`, `connect()` e `accept()` devono poter accettare **qualsiasi tipo di indirizzo** (IPv4, IPv6, Unix Local, ecc.). Tuttavia, il C non supporta il polimorfismo o l'ereditarietà: una funzione non può accettare tipi di dato diversi nello stesso parametro. Per risolvere questo limite, gli ingegneri di Berkeley hanno creato la `struct sockaddr` (definita in `<sys/socket.h>`). Che è una sorta di prototipo generale, poi esistono varie specifiche.

```
/*
 * Address families.
 */
#define AF_UNSPEC 0      /* unspecified */
#define AF_LOCAL  1      /* local to host (pipes, portals) */
#define AF_UNIX   AF_LOCAL /* backward compatibility */
#define AF_INET   2      /* internet: UDP, TCP, etc. */
#define AF_IMPLINK 3      /* arpanet imp addresses */
//....
#define AF_APPLETALK 16    /* Apple Talk */
#define AF_ROUTE     17    /* Internal Routing Protocol */
#define AF_LINK      18    /* Link layer interface */
//....
#define AF_INET6     28    /* IPv6 */
#define AF_NATM      29    /* native ATM access */
#define AF_ATM       30    /* ATM */
//....
```

Usiamo la struttura specifica per IPv4: `struct sockaddr_in` (che ha i campi comodi `sin_port` e `sin_addr`

```
/*
 * Socket address, internet style.
 */
struct sockaddr_in {
    uint8_t    sin_len;
    sa_family_t sin_family;
    in_port_t  sin_port;
    struct in_addr sin_addr;
    char       sin_zero[8];
};
```



I campi più importanti sono `sin_family`, `sin_addr` e `sin_port`.

Immaginiamo di voler scrivere un client che richieda l'ora esatta ad un server, vogliamo usare TCP/IP quindi dobbiamo specificare `AF_INET` nel campo `sin_family` →

```
sa.sin_family = AF_INET;
```

Network byte order

Dobbiamo considerare anche il problema dell'endianess, alcuni computer usano il little altri il big endian quindi i dati devono essere convertiti seguendo il network byte order prima di essere spediti. Oppure possono essere mandati come stringa perché le stringhe vengono sempre codificate in ASCII quindi non sono interessate dall'endianess.

- *Seguire il network byte order*
 - **h** = **Host** (il tuo computer)
 - **to** = **to** (verso)
 - **n** = **Network** (la rete Internet)
 - **s / l** = **Short** (16 bit) / **Long** (32 bit)

Htonl si usa per convertire gli indirizzi IPv4 (essendo numeri da 32 bit)

Htons si usa per convertire le porte (essendo numeri da 16 bit)

- *Inviare una stringa*
 - si usa `inet_addr` → `sa.sin_addr.s_addr = inet_addr("193.204.114.105");`

Funzione connect()

Quando un client ha creato la socket deve connettersi con una macchina remota su una porta specifica. Usa connect().

```
int connect(int s, const struct sockaddr *name, socklen_t namelen);
```

Il parametro `int s` è il valore di ritorno della funzione socket()

Il parametro `const struct sockaddr * name` è il puntatore alla struttura sockaddr

Il parametro `socklen_t namelen` riguarda il numero di byte all'interno di sockaddr (il secondo parametro)

Se la connect() ha avuto successo ritorna 0 altrimenti ritorna -1 e salva il valore di errore all'interno di `errno`

Socket lato server e funzione bind()

Il server solitamente aspetta che qualche client lo contatti, ci sono 65535 porte ma di solito un server processa le richieste che arrivano solo su una di queste, usiamo le

socket per dire a quale porta vogliamo essere contattati. Per questo si usa la funzione `bind()`.

```
int bind(int s, const struct sockaddr *addr, socklen_t addrlen);
```

Il parametro `int s` è il valore di ritorno della funzione `socket()`

Il parametro `const struct sockaddr * addr` è il puntatore alla struttura `sockaddr`

Il parametro `socklen_t namelen` riguarda il numero di byte all'interno di `sockaddr` (il secondo parametro)

Se la `bind()` ha avuto successo ritorna 0 altrimenti ritorna -1

La funzione `bind()` serve quindi a **legare** (assegnare) un indirizzo IP e un numero di porta al socket che hai creato.

Oltre a specificare la porta all'interno dell'indirizzo (`addr`), il server può includere il proprio indirizzo IP. Tuttavia, può semplicemente utilizzare la costante simbolica `INADDR_ANY` per indicare che servirà tutte le richieste dirette alla porta specificata, indipendentemente da quale sia il suo indirizzo IP. Questo simbolo, insieme a diversi altri simili, è dichiarato nel file `<netinet/in.h>`.

Example

Se nel codice inserisci un IP specifico, il tuo server accetterà connessioni **solo** da quella specifica rete.

Usando invece `INADDR_ANY`, dici al sistema operativo: *"Non mi importa da quale scheda di rete arrivi la richiesta (Wi-Fi, cavo Ethernet o locale), se qualcuno bussa alla porta 8080, tu passami la connessione"*. È il modo standard per rendere un server accessibile da ovunque.

Funzione `listen()`

Il server garantisce l'ascolto su quella porta tramite la funzione `listen`. Più precisamente **Cambia lo stato del socket**: Dice al sistema operativo che quel socket non è più un socket normale (attivo, che avvia connessioni), ma diventa un socket **passivo** (che può solo ricevere chiamate).

```
int listen(int s, int backlog);
```

Il parametro `int s` è il valore di ritorno della funzione `socket()`

Il parametro `backlog` è un numero interno che definisce la lunghezza massima della coda delle connessioni in attesa

Funzione accept()

Il server accetta la connessione tramite la funzione `accept()`.

Nota che questa volta `addrlen` è un **puntatore**. Questo è necessario perché in questo caso è la socket che compila `addr`, ovvero la struttura `sockaddr_in`. Il valore di ritorno è un intero. Infatti, la `accept` restituisce una **nuova socket**. Userai questa nuova socket per comunicare con il client. Cosa succede alla vecchia socket? Continua a rimanere in ascolto per ulteriori richieste (ricordi la variabile `backlog` che abbiamo passato alla `listen()` finché non la chiudiamo).

```
int accept(int s, struct sockaddr *addr, socklen_t *addrlen);
```

Il parametro `int s` è il valore di ritorno della funzione `socket()`.

Il parametro `const struct sockaddr * addr` è il puntatore alla struttura `sockaddr`.

Il parametro `socklen_t * addrlen` è un **puntatore a un intero** che rappresenta la dimensione della struttura dell'indirizzo.

- **Come funziona:** Viene passato "per riferimento" (usando l'operatore `&`, es. `&addr_len`).
- **A cosa serve:** È un parametro di *Input/Output* (chiamato anche *Value-Result*).
 - **In Input:** Dice al sistema operativo quanto spazio massimo ha a disposizione nella struttura `addr` che gli stiamo passando.
 - **In Output:** Poiché è la `accept()` che compila la struttura con i dati del client che si è appena connesso, prima di terminare la funzione **sovrascrive** questa variabile inserendo il numero esatto di byte che ha realmente utilizzato per scrivere l'indirizzo del client

Se la funzione `accept()` ritorna un numero intero ≥ 0 (Successo): Rappresenta un **nuovo File Descriptor** (ovvero una nuova socket di connessione). Altrimenti ritorna -1.

Funzioni send e receive

Send

```
send(int sockfd, const void *buf, size_t len, int flags);
```

Invia i dati contenuti in un buffer di memoria attraverso il socket specificato.

- `int sockfd`: Il descrittore del socket (es. il tuo `socketFD`). È l'ID della connessione aperta verso l'altro computer.
- `const void *buf`: Il puntatore alla memoria dove risiedono i dati che vuoi spedire (nel tuo codice, la variabile `buffer`). Può essere del testo, un file, o qualsiasi tipo di dato.

- **size_t len**: La quantità esatta di byte che vuoi inviare. Nel tuo codice hai usato **strlen(buffer)**, che è perfetto per le stringhe perché calcola quanti caratteri ci sono prima del **\0**.
- **int flags**: Opzioni speciali per l'invio. Nella stragrande maggioranza dei casi si passa **0**, che indica un comportamento standard (bloccante finché i dati non vengono presi in carico dal sistema).

Valore di ritorno: Restituisce il numero di byte *effettivamente* inviati

Receive

```
recv(int sockfd, void *buf, size_t len, int flags);
```

Riceve i dati che arrivano dal socket e li copia all'interno di un buffer nel tuo programma. Di base, **recv** è una **funzione bloccante**: se non ci sono dati in arrivo, il thread che la esegue si "addormenta" su quella riga e si risveglia solo quando l'altro computer invia qualcosa (o si disconnette).

- **int sockfd**: Il descrittore del socket da cui vuoi "ascoltare" e prelevare i dati in arrivo.
- **void *buf**: Il puntatore all'area di memoria (il buffer) dove la funzione deve *scrivere* i byte che riceve dalla rete.
- **size_t len**: La dimensione massima del tuo buffer (la capienza massima del "secchio" in cui raccogli i dati, ad esempio **BUFFER_MAX**). Serve a evitare che **recv** scriva più dati di quanti la memoria possa contenerne, causando un crash.
- **int flags**: Opzioni speciali per la ricezione. Anche qui, di solito si passa **0** per il comportamento standard.

****Valore di ritorno:** È fondamentale per capire lo stato della connessione:

- **Maggiore di 0**: Indica il numero di byte effettivamente ricevuti.
- **Uguale a 0**: Significa che **l'altro computer ha chiuso la connessione** ordinatamente (ha fatto la **close**).
- **Uguale a -1**: Significa che si è verificato un errore di rete.

SOCKET DATAGRAM

Funzioni sendto() e recvfrom()

```
sendto(int sockfd, const void *buf, size_t len, int flags, const struct
sockaddr *dest_addr, socklen_t addrlen)
```

Serve ad inviare messaggi con UDP.

Il parametro `int sockfd` è il file descriptor della socket creata precedentemente con la funzione `socket()`.

Il parametro `const void *buf` è un puntatore al buffer di memoria che contiene i dati da inviare.

Il parametro `size_t len` rappresenta la quantità di byte che si vogliono mandare.

Il parametro `int flags` sono dei flag per delle opzioni particolari.

Il parametro `const struct sockaddr *dest_addr` è un puntatore alla struttura che contiene indirizzo IP e porta del destinatario.

Il parametro `socklen_t addrlen` rappresenta la dimensione della struttura passata precedentemente.

Valore di ritorno: in caso di successo ritorna il numero di byte inviati, se fallisce ritorna -1.

```
recvfrom(int sockfd, void *buf, size_t len, int flags, struct sockaddr
*src_addr, socklen_t *addrlen)
```

Questa è la esatta controparte della funzione `sendto`. La `recvfrom` è bloccante.

Il parametro `int sockfd` è il file descriptor della socket creata precedentemente con la funzione `socket()`.

Il parametro `void *buf` è un puntatore al buffer di memoria in cui verranno scritti i dati.

Il parametro `size_t len` rappresenta la dimensione massima in byte del buffer.

Il parametro `int flags` sono dei flag per delle opzioni particolari.

Il parametro `struct sockaddr *src_addr` è un puntatore a una struttura vuota. La funzione **la riempirà automaticamente** con l'indirizzo IP e la porta del mittente. Se non ti interessa sapere chi ha inviato il dato, puoi passare `NULL`.

Il parametro `socklen_t *addrlen`, **attenzione, questo è un puntatore**, prima della chiamata deve contenere la dimensione della struttura passata in `src_addr`. Al ritorno dalla funzione, conterrà la dimensione effettiva dell'indirizzo memorizzato.

Valore di ritorno: in caso di successo ritorna il numero di byte ricevuti altrimenti ritorna -1.

Buona Prassi Di Programmazione

Questo discorso vale sia per socket TCP che per socket UDP

1. Quando si usano le struct nei socket?

Le struct si usano solitamente quando devi scambiare **più dati eterogenei contemporaneamente** (es. un intero, tre float e una stringa) in un'unica spedizione (`send`), senza dover fare mille chiamate a funzioni di rete separate.

Raggruppare tutto in un unico "pacchetto" binario rende il codice più pulito e riduce il carico sulla rete.

2. Le struct devono essere IDENTICHE tra Client e Server?

Se utilizziamo questo approccio (ovvero mandare tutta la struct in un unica send) allora **si** le struct devono essere identiche sia lato client che lato server, anche se magari nella logica del server un campo della struct non viene utilizzato.

(Per evitare che client e server abbiano la stessa struct anche se non serve, i programmatori o fanno più send per inviare i dati oppure mandano tutti i dati come un testo unico e poi separano tutti i dati a destinazione, ma per noi va bene anche usare 2 struct identiche in ogni caso).